Algoritmos e Estruturas de Dados

1ª Série

(Problema)

Análise de ocorrência de menções em tweets

N. 51447 Ricardo Pinto

N. 51445 José Saldanha

N. 51484 Vasco Pereira

Licenciatura em Engenharia Informática e de Computadores

Semestre de Verão 2023/2024

16/4/2023

Índice

Conteúdo

1. INTRODUÇÃO **2**

2. ANÁLISE DE OCORRÊNCIA DE MENÇÕES EM TWEETS **2**

2.1 ANÁLISE DO PROBLEMA 2

2.2 ESTRUTURAS DE DADOS 3

2.3 ALGORITMOS E ANÁLISE DA COMPLEXIDADE 4

3. AVALIAÇÃO EXPERIMENTAL **6**

4. CONCLUSÕES **8**

REFERÊNCIAS **11**

1. Introdução

Neste relatório, vamos abordar o tema do problema da 1º série: análise de ocorrência de menções em *tweets*. Para este problema, pretendeu-se desenvolver uma aplicação que permite analisar as menções a jogadores ou outras menções relacionadas com as seleções durante o mundial de futebol do Brasil, em 2014.

Os principais objetivos do problema é vermos qual *hastag* é a mais mencionada dado um conjunto de *tweets* e ainda ver quantos *tweets* encontram-se mais perto de uma determinada data.

O relatório está dividido em 6 partes:

- Aprofundação sobre o que é o problema e como foi abordado;
- Avaliação experimental do problema;
- Conclusões tiradas;
- Resolução do 2 conjunto de perguntas;
- Referências utilizadas para o relatório.

2. Análise de ocorrência de menções em tweets

Resumindo o conteúdo da secção 2, na secção 2.1 é descrito o problema e a abordagem encontrada para o resolver. Na secção 2.2 apresentam-se as estruturas de dados utilizadas. Por fim, a secção 2.3 contém alguns dos principais algoritmos utilizados, como os mesmos consultam/atualizam as estruturas de dados e a análise da sua complexidade.

2.1 Análise do problema

Para este problema, vamos considerar que cada *tweet* (ou o seu excerto) é uma *string* que contém: a data em que foi publicado, um conjunto de *hastags* agregado ao *tweet*, o *id* do *tweet* e o *uid* (identificador de utilizador) de quem publicou o *tweet*. Assim, teremos um conjunto de *tweets* composto pela data de criação, um conjunto de *hastags*, o *id* e o *uid* e ainda um conjunto de *hastags* em que pelo menos uma está incluída num ou mais *tweets* do conjunto anterior.

Para retirarmos os *tweets* do ficheiro e armazená-los no programa iremos utilizar uma função que leia o conteúdo e armazene os dados em algum lado. Para isto utilizámos uma data class com o nome Twts. Depois criamos uma lista com esta data class cujo cada elemento correspondente vai para a mesma e, depois, vai para uma lista.

Para contarmos as menções decidimos fazer algo parecido com a função que lê os conteúdos dos *tweets*, mas em vez de armazenar logo numa lista com a *data class* correspondente a *data class Hastags*, que contém uma *string* com a *hastag* e uma variável que serve para contar as menções. Decidimos, então, que por cada linha lida no ficheiro que contém as *hastags*, ele verifica os elementos do conjunto de *hastags* que pertence à lista de *tweets* e sempre que esse conjunto contém a *hastag* a ser analisada o valor para o contador incrementa em 1.

Para verificar quantos *tweets* encontram-se perto de uma certa data, decidimos usar uma função que converte a *string* correspondente à data num número do tipo *Long*. De seguida, decidimos transformar a lista de *tweets* num *array* de pares que contém os *tweets* e ainda o módulo da diferença entre a data a que foram criados os *tweets* e a data escolhida pelo utilizador, sendo tudo convertido para um número. Para ordenar os *tweets* decidimos utilizar o algoritmo *heapSort* em relação ao segundo elemento do par incluído por cada elemento do *array*, ou seja, aplicado ao valor do módulo da diferença entre a data a que foram criados os tweets e a data escolhida pelo utilizador.

2.2 Estruturas de Dados

As estruturas de dados utilizadas para o problema foram a lista e o *array*. Ambos se assemelham pela forma como está apresentada na figura 1. Para as listas usámos listas mutáveis, uma vez que nós queríamos adicionar novos elementos à lista sempre que possível, o que não é possível com listas imutáveis.

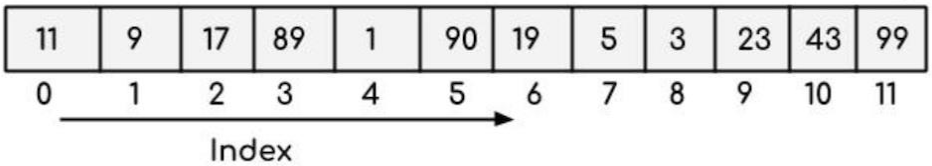

Figura 1: Exemplo da estrutura de dados *array*.

Para além das listas e dos *arrays*, ainda utilizamos *data class*, que organiza diferentes objetos dentro de um grupo. Na figura abaixo, figura 2, encontra-se uma exemplificação de como é uma *data class* e o seu uso.

```
data class Record(var name: String, var roll: Int)

fun main()
{
val r1 = Record("Ram", 8)
println("data of object r1 : $r1")

val r2 = r1.copy()
println("data of cpied object r2: $r2")

val r3 = r1.copy(roll = 10)
println("Modified data of r1 : $r3")
}
```

Figura 2: Exemplificação de uma *data class* e como pode ser utilizada.

2.3 Algoritmos e análise da complexidade

Os algoritmos utilizados, tanto para ler o ficheiro dos *tweets*, tanto para contar as menções são relativamente parecidos. Temos um ciclo *forEachLine*, que funciona como um ciclo *for*, ou seja, possui uma complexidade temporal igual a *O(n)*. Isto faz com que no caso da função que lê os *tweets*, por cada linha que o programa lê ele executa um conjunto de operações, que neste caso são as operações de separação de conteúdo e, depois, adiciona esse conteúdo a uma lista sob uma *data class*. Esta função ainda tem uma complexidade espacial igual a *O(n)*, uma vez que ele executa esta código consoante o número de linhas que o ficheiro que contem os *tweets*. A figura 3 exemplifica como isto acontece.

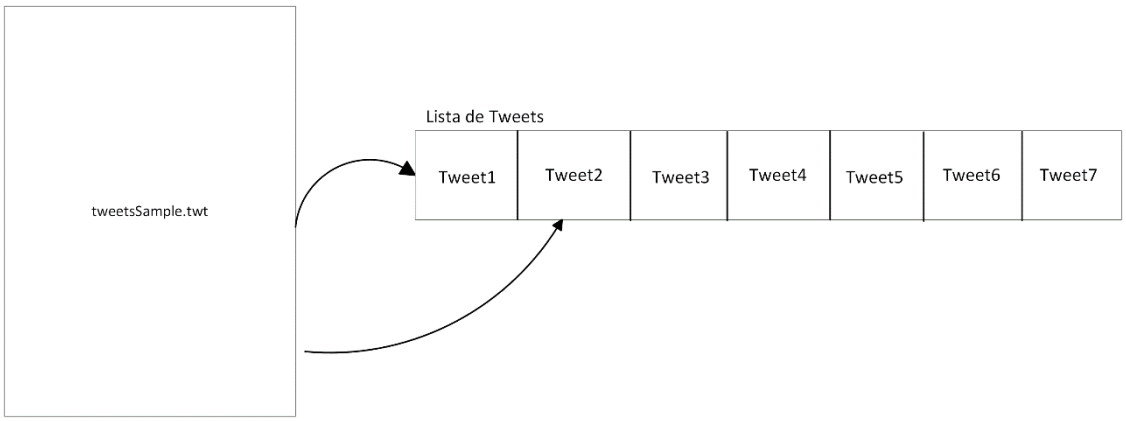

Figura 3: Exemplificação de como ocorre a transferência do ficheiro para a lista.

Para a função que conta as menções, exibimos o mesmo algoritmo, mas com a diferença de que para além de ele analisar cada linha do ficheiro das *hastags*, o programa vai analisar cada conjunto de *hastags* da lista de *tweets* e verifica se a *hastag* que está a ser analisada está contida nesse mesmo conjunto para no fim incrementar o contador em 1. Assim, a complexidade temporal desta função é *O(n*m)*, uma vez que ele analisa cada linha do ficheiro das *hastags* e ainda analisa todos os elementos da lista de *tweets*. Na figura 4, podemos ver um exemplo de como isto ocorre.

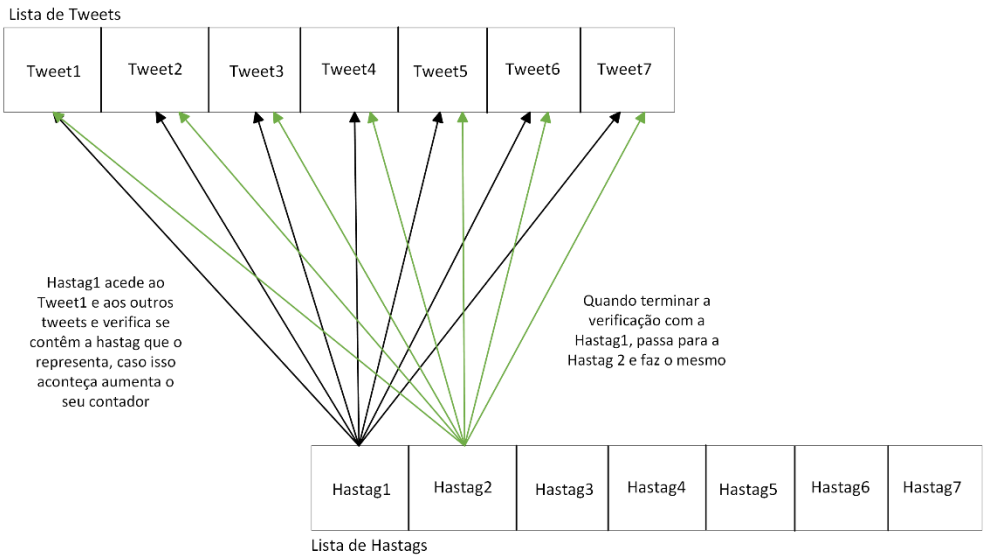

Figura 4: Exemplificação de como a lista de *hastags* acede e aumenta o seu contador.

Para a função que vê os k *tweets*, decidimos utilizar o *heapSort* para ordenar os elementos. Como o nosso array é composto por elementos de um par entre os *tweets* e o módulo da diferença entre a data a que foram criados os tweets e a data escolhida pelo utilizador, precisámos de fazer três pequenas alterações ao código apresentado em tempo de aulas. Em vez de o algoritmo usar *IntArray*, ele utiliza *Array<Pair<Twts, Long>>* e as outras duas alterações ocorrem na função *maxHeapify*, que em vez de ler o conteúdo todo do *array*, só vai ler o segundo elemento do par. Isso originou uma troca de *array[l]>array[pai]* para *array[l].second>array[pai].second*. Assim, a complexidade temporal do nosso algoritmo para esta função assume a complexidade temporal do *heapSort*, que é $O(n \log_2 n)$, e a sua complexidade espacial é $O(n)$. A figura 5 exemplifica como o *heapSort* em *maxHeap* funciona.

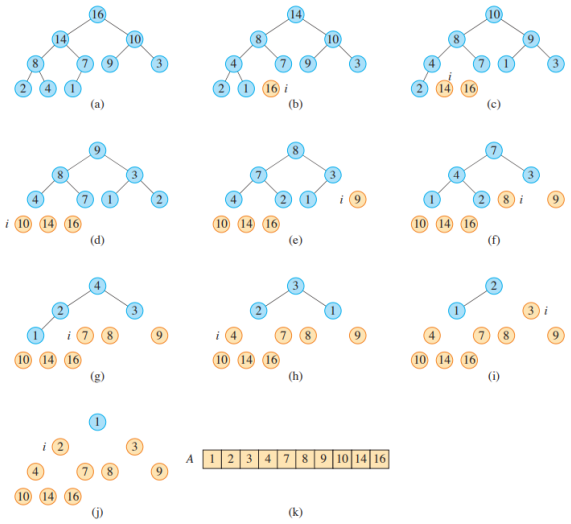

Figura 5: Exemplo de atualização de uma estrutura de dados (ordenação de modo crescente dos dados utilizando o algoritmo *heapSort*).

3. Avaliação Experimental

Para a avaliação experimental do problema, usámos o computador do Ricardo que é um portátil HP com um processador Intel i7 da 12ª geração e com 16 GB de RAM. Foram feitos 10 testes para cada função e para cada mudança de valor a testar, ou seja, quando tivemos a testar a função *countMentions* com 10 *hastags*, fizemos dez testes para essa função e obtivemos os seguintes resultados:

3.1 *countMentions*

	Tempos de Execução (ms)									
10 Hastags	1503	1538	1455	1487	1403	1548	1436	1369	1693	1537
15 Hastags	1565	1549	1613	1596	1531	1563	1554	1700	1419	1345
25 Hastags	1696	1960	1974	2222	2054	1844	1935	1937	2150	1037

Tabela 1: Resultados do tempo de execução da função *countMentions*, a média corresponde a 1497 ms para 10 *hastags*, 1543 ms para 15 *hastags* e 1881 ms para 25 *hastags*.

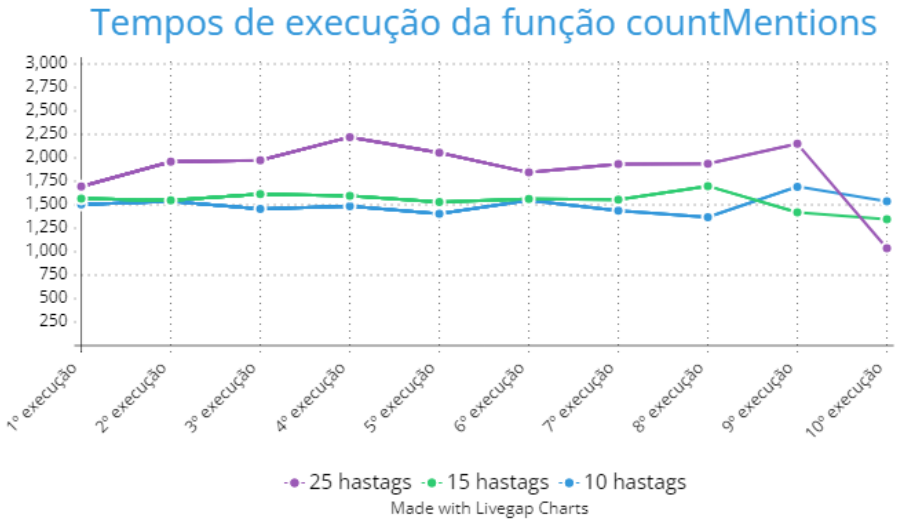

Gráfico 1: Gráfico correspondente à tabela 1.

3.2 *nearDate* - 2014-07-14 03:16:07

	Tempos de Execução (ms)									
100 mil *Tweets*	3506	3443	3575	3475	3488	3400	3575	3388	3388	3452
500 mil *Tweets*	3521	3624	3552	3394	3505	3406	3416	3822	3544	3413
1 milhão *Tweets*	3589	3487	3452	3511	3460	3434	3498	3423	3561	3914

Tabela 2: Resultados do tempo de execução da função *nearDate* com a data "2014-07-14 03:16:07". A média corresponde a 3469 ms para 100 mil *tweets*, a 3520 ms para 500 mil *tweets* e a 3533 ms para 1 milhão *tweets*

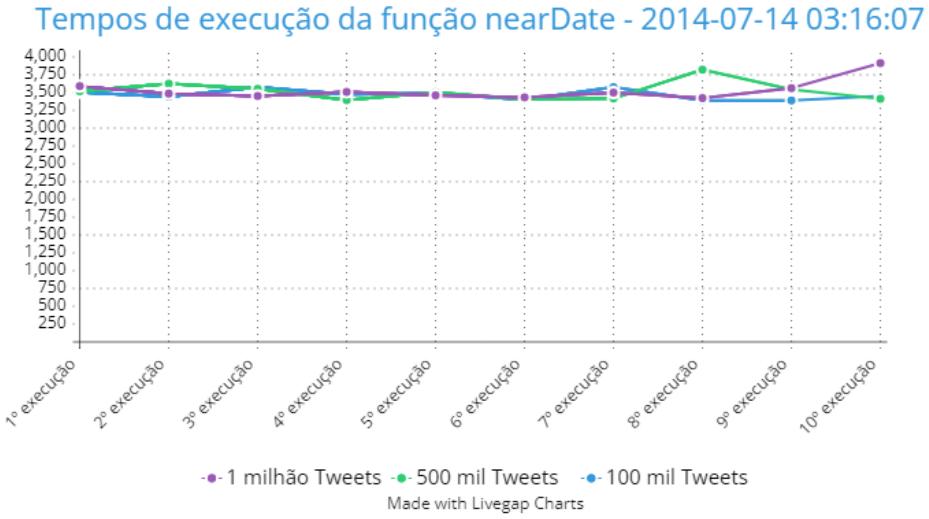

Gráfico 2: Gráfico referente aos resultados da tabela 2.

3.3 *nearDate* - 2014-07-15 15:05:01

	Tempos de Execução (ms)									
100 mil *Tweets*	3327	3289	3263	3217	3371	3149	3117	3138	3150	3244
500 mil *Tweets*	3253	3465	3446	3195	3262	3252	3366	3136	3333	3204
1 milhão *Tweets*	3361	3386	3489	3244	3530	3396	3432	3224	3257	3415

Tabela 3: Resultados do tempo de execução da função *nearDate*, com a data "2014-07-15 15:05:01". A média corresponde a 3226 ms para 100 mil *tweets*, a 3291 ms para 500 mil *tweets* e a 3373 ms para 1 milhão *tweet*.

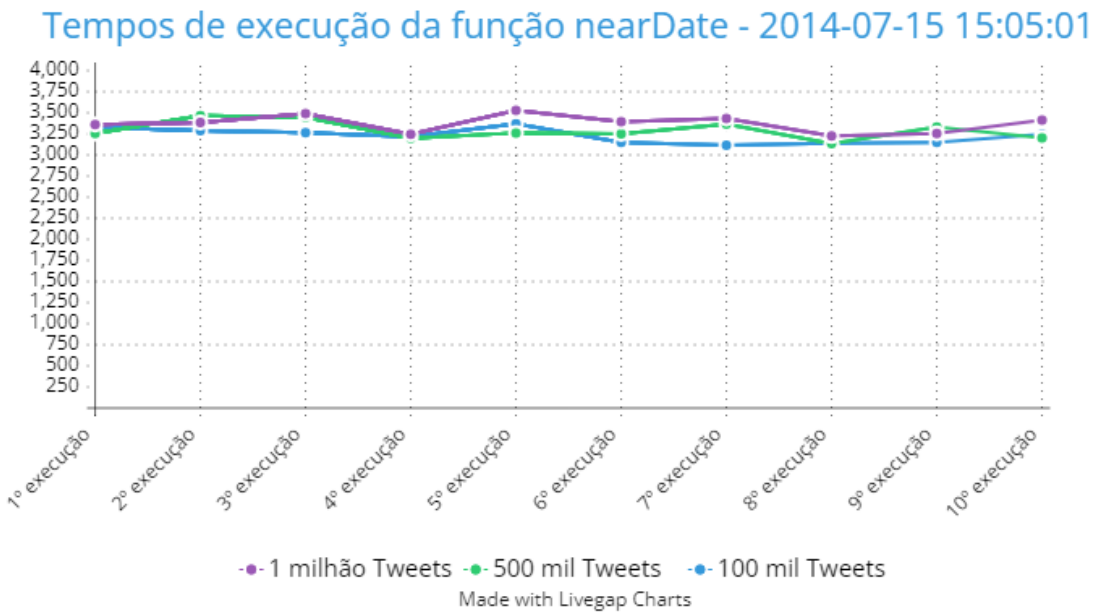

Gráfico 3: Gráfico referente aos resultados da tabela 3.

4. Conclusões

Concluindo, podemos ver através da análise dos tempos de execução que a nossa implementação é uma implementação que até certo ponto é otimizada, uma vez que os tempos de execução são baixos. Comparando os tempos de execução com os custos temporais referidos no ponto 2.3, concordamos que estes estejam em acordo. Contudo, soubemos que existe uma maneira de fazer com que a função *nearDate* tenha um custo temporal ainda menor do que aquele que nós implementámos, ou seja, em vez de ter um custo $O(n \log_2 n)$, tem um custo $O(n \log_2 k)$, em que k é o número de tweets mais próximos de uma certa data, portanto, assumimos este custo maior que implementámos, mesmo não sendo a forma mais eficiente do algoritmo.

Com isto pudemos aprender a como fazer algoritmos e tentar fazer com que os mesmos tenham custo temporal menor do que o idealizado e implementado inicialmente.

5. Resolução do Exercício 2

1

$$\boxed{C(n) = \begin{cases} O(1), & n <= 1 \\ O\left(\frac{n}{2}\right) + C\left(\frac{n}{2}\right), & n > 1 \end{cases}} \rightarrow \text{Equação de recorrência}$$

$$C(n) = O\left(\frac{n}{2}\right) + C\left(\frac{n}{2}\right)$$

$$= O\left(\frac{n}{2}\right) + \left(O\left(\frac{n}{2^2}\right) + C\left(\frac{n}{2^2}\right)\right)$$

$$= O\left(\frac{n}{2}\right) + O\left(\frac{n}{2^2}\right) + \left(O\left(\frac{n}{2^3}\right) + C\left(\frac{n}{2^3}\right)\right)$$

$$n = 2^m \Leftrightarrow m = \log_2 n$$

$$C(2^m) = \begin{cases} O(1), & m = 0 \\ O(2^{m-1}) + C(2^{m-1}), & m > 0 \end{cases}$$

$$C(2^m) = O(2^{m-1}) + C(2^{m-1})$$

$$= O(2^{m-1}) + (O(2^{m-2}) + C(2^{m-2}))$$

$$= O(2^{m-1}) + O(2^{m-2}) + (O(2^{m-3}) + C(2^{m-3}))$$

$$\ldots$$

$$= O(2^{m-1}) + O(2^{m-2}) + O(2^{m-3}) + \ldots O(2^{m-3}) + C(2^{m-m})$$

$$= O\left(\sum_{i=0}^{m-1} 2^i\right) + O(1) = O(2^m - 1) + O(1) = O(2^m)$$

Como $2^m = n \rightarrow O(2^m) = \boxed{O(n)}$

Resposta: A complexidade é $O(n)$

② 2.1)

$$C(n) = \begin{cases} O(1) & , \text{se } n = 0 \\ O(1) + 2C\left(\frac{n}{2}\right) & , \text{se } n \neq 0 \end{cases}$$

Equação de recorrência

$$C(n) = O(1) + 2C\left(\frac{n}{2}\right)$$
$$= O(1) + 2\left(O(1) + 2C\left(\frac{n}{2^2}\right)\right)$$
$$= O(1) + O(1) + 2^2 C\left(\frac{n}{2^2}\right)$$
$$= O(1) + O(1) + 2^2\left(O(1) + 2C\left(\frac{n}{2^3}\right)\right)$$
$$= O(1) + O(1) + O(1) + 2^3 C\left(\frac{n}{2^3}\right)$$

...

Seja $\frac{n}{2^i} = 1 \Leftrightarrow n = 2^i \Leftrightarrow i = \log_2 n$, então

$$C(n) = i \times O(1) + 2^i C\left(\frac{n}{2^i}\right)$$
$$= O(i) + 2^i O(1)$$
$$= O(\log_2 n) + n\, O(1)$$
$$= O(\log_2 n) + O(n) = \boxed{O(n)}$$

Complexidade Temporal

2.2) É sim possível diminuir a complexidade de $O(n)$ para $O(\log_2 n)$ atribuindo uma val para xpto $(x, n/2)$. Assim o cálculo recursivo passa a ser feito apenas uma vez, diminuindo a complexidade temporal de $O(n)$ para apenas $O(\log_2 n)$ (função de custo temporal logarítmico).

3) Analisando a função count ignorando, inicialmente, a chamada do algoritmo de ordenação sort, concluímos que possui complexidade temporal $O(n \log_2 n)$, visto que o ciclo for, nesta função, tem complexidade $O(n)$ e a chamada à binarySearch, que é feita dentro do loop, tem complexidade $O(\log_2 n)$ pelo que, juntando estas duas complexidades, ficamos com uma complexidade temporal de $O(n \log_2 n)$. Assim, sort tem de ser obrigatoriamente igual ou inferior a $O(n \log_2 n)$. Existem 2 algoritmos de ordenação possíveis para se usar: mergeSort e heapSort, ambos com complexidade temporal $O(n \log_2 n)$.

Para escolher, então, qual deles usar, olhamos para a complexidade espacial de cada algoritmo e estabilidade dos mesmos. Tendo em vista estes 2 parâmetros para esta função decidimos escolher o mergeSort pois apesar de maior complexidade espacial, $O(n)$, é muito mais estável que o heapSort, que é um algoritmo instável ~~não que~~ mesmo tendo complexidade espacial $O(1)$.

Referências

[1] “Disciplina: Algoritmos e Estruturas de Dados - 2324SV,” Moodle 2023/24. [Online]. Available: https://2324.moodle.isel.pt. [Accessed: 07-04-2024].

[2] Introduction to Algorithms, 3º Edition. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. MIT Press.